MILESTONE 2 • 3 MARKS

Project Requirements Presentation

JAVA DESKTOP APPLICATION • Swing UI • JDBC • Database

PROJECT TITLE

Exam Seat Arrangement System

Team No.: _____ | Class/Semester: _____ | Review Date: _____

MANDATORY EVIDENCE

Team and HackerRank Java progress

Update this table in every milestone submission and PPT

#	Student name	Roll no.	Role	Profile / username	Previous Java stars	Current Java stars	Progress
1	**Antony Zavior K**	62	Team Lead	https://www.hackerrank.com/profile/haxey61790	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	
2	**Lijo P Thomas**	67	Member	https://www.hackerrank.com/profile/lijot707	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	
3	**Milind Krishnan**	69	Member				

REQUIRED Show the profile URL or username and record Java stars (0–5) as on the review date. Every member must demonstrate continuing Java practice.
Recommended evidence: a current HackerRank profile screenshot in the appendix or a profile link in this table.

The problem and its context

Make the need for the application clear before describing features

EXISTING PROCESS

- Exam cell staff prepare seating plans by hand on paper or in spreadsheets for every exam session.
- Hall-wise lists are printed and pasted on notice boards.

AFFECTED USERS

- Primary: exam coordinator and exam cell staff who prepare the plan.
- Secondary: students, invigilators and department heads who depend on it.

PAIN POINTS

- Hours of effort per exam series; seats double-booked or halls overfilled.
- Same-branch students end up side by side, and any last-minute change means redoing the plan.

PROBLEM STATEMENT

Manual exam seat allocation is slow and error-prone, and exam coordinators and staff have no reliable way to prevent clashes or update plans quickly. As a result, students may be wrongly seated and malpractice becomes easier, while last-minute changes cause confusion on exam day.

Objectives and project scope

Separate expected outcomes from features and implementation choices

PROJECT OBJECTIVES

1. Produce a clash-free seating plan for an exam session in minutes instead of hours.
2. Keep halls, students, exams and allocations in one central database with no duplication.
3. Provide hall-wise seating lists, attendance sheets and quick seat lookup for students and staff.
4. Apply Java OOP, Swing GUI, JDBC and exception handling in a well-structured application.

SCOPE BOUNDARY

IN SCOPE

- Hall, student and exam management
- Automatic seat allocation with same-branch separation
- Seating grid with manual swap
- Seat search and hall-wise reports
- Input validation and error handling

OUT OF SCOPE

- Web or mobile version
- SMS / email notifications to students
- Integration with the university portal
- Single-machine use with one MySQL server

Users, responsibilities and system flow

Who performs each major operation and how data moves through the application

01

Exam coordinator (Admin)

Manages halls, students, exams; runs allocation; prints reports

02

Invigilator / staff

Views assigned hall chart and attendance sheet

03

Student / help desk

Looks up hall and seat by register number

IMPROVEMENT OVER CURRENT PROCESS

Seating plan in minutes, no double-booked seats, and any change re-run in seconds.

SYSTEM FLOW

01 User action
Select exam and halls, click Allocate

→

02 Swing UI
Allocation screen and seating grid

→

03 Java logic
Validate input, apply allocation rules

→

04 JDBC
INSERT / SELECT / UPDATE statements

→

05 Database
Halls, students, exams, allocations

PROPOSED SOLUTION A Java Swing desktop application with JDBC and MySQL lets the exam coordinator enter halls, exams and students and generate the seating plan automatically. Seats are allocated hall by hall without double-booking, keeping students of the same branch apart. Results are shown as a seating grid and can be searched and printed as reports.

What the system shall do

Specific • Necessary • Feasible • Testable • Traceable to a user or module

Req. ID	Requirement statement	Actor	Priority	Acceptance check
FR-01	The system shall authenticate admin and staff by username and password before access.	Admin, Staff	**H**	Wrong password is rejected; correct one opens the dashboard.
FR-02	The system shall add, edit and delete halls with rows, columns and seats per bench.	Admin	**H**	Hall capacity equals rows × columns × seats per bench.
FR-03	The system shall maintain student records and import them from a CSV file.	Admin	**H**	Imported count matches CSV rows; duplicate register numbers rejected.
FR-04	The system shall create exam sessions and assign invigilators to halls.	Admin	**M**	Session saved with date, time and hall staff; visible in the list.
FR-05	The system shall automatically allocate seats for a session without double-booking, keeping same-branch students apart.	Admin	**H**	No seat holds two students; adjacent seats differ in branch on test data.
FR-06	The system shall show each hall as a seating grid and allow a manual seat swap.	Admin, Invigilator	**M**	Grid matches the database; swap updates both students’ seats.
FR-07	The system shall find a student’s hall and seat by register number or name.	Student, Admin	**H**	Known register number returns the correct hall and seat.
FR-08	The system shall generate hall-wise seating lists and attendance sheets for print or export.	Admin, Invigilator	**M**	Report lists every allocated student of the hall in seat order.

Major modules and proposed Swing screens

Every functional requirement belongs to a module

M1

Login & Access

CORE FUNCTIONS

- Secure login
- Role check (admin / staff)
- Logout and session end

Swing screens: LoginFrame, DashboardFrame

Related FRs: FR-01

M2

Master Data

CORE FUNCTIONS

- Manage halls and layouts
- Manage students, CSV import
- Manage exams, invigilators

Swing screens: HallPanel, StudentPanel, ExamPanel

Related FRs: FR-02, FR-03, FR-04

M3

Seat Allocation

CORE FUNCTIONS

- Auto-allocate for a session
- Same-branch separation
- Grid view and manual swap

Swing screens: AllocationPanel, SeatingGridPanel

Related FRs: FR-05, FR-06

M4

Search & Reports

CORE FUNCTIONS

- Seat search
- Hall-wise seating list
- Attendance sheet export

Swing screens: SearchPanel, ReportPanel

Related FRs: FR-07, FR-08

Database and input/output requirements

Identify what must be stored and validated; detailed schema comes in the design milestone

Entity / data	Key information to store	Relationship / owner	CRUD needed
User	Username, password hash, role	Admin / staff account	C, R, U
Hall	Hall name, rows, columns, seats per bench	Has many seats in allocations	C, R, U, D
Student	Register no., name, branch, semester	Appears in allocations	C, R, U, D
Exam	Subject, date, session, invigilator	Uses halls and students	C, R, U, D
Allocation	Exam, student, hall, row, column	Links Exam, Student, Hall	C, R, U, D

INPUT VALIDATION

- Mandatory fields
- Data type and range
- Date / format checks
- Duplicate register numbers
- Meaningful error messages

IMPORTANT INPUTS

- Hall, student and exam forms
- CSV file of students
- Register number search
- Exam and hall selection

EXPECTED OUTPUTS

- Seating grid per hall
- Hall-wise seating list
- Attendance sheet
- Confirmations and error dialogs

Non-functional requirements and technology constraints

Quality expectations turned into measurable acceptance conditions

USABILITY

Clear labelled forms; any main screen reachable within 3 clicks.

RELIABILITY

Saved data is retrieved exactly; allocation runs as one transaction (all or nothing).

PERFORMANCE

Allocate about 500 students within 5 seconds; searches return within 1 second.

VALIDATION

Empty, non-numeric, duplicate and invalid-date inputs are rejected with a message.

ERROR HANDLING

Friendly dialog and safe rollback; no crash if the database is unreachable.

MAINTAINABILITY

Layered packages (ui, model, dao, service), clear names and JavaDoc comments.

MANDATORY TECHNOLOGY BASELINE

LANGUAGE	DESKTOP UI	CONNECTIVITY	DATABASE	IDE
Java (JDK 17)	Swing	JDBC	MySQL 8.0	IntelliJ IDEA

REVIEW SUMMARY

Requirements baseline

Confirm the scope before moving to project design

MILESTONE EVIDENCE CHECKLIST

- ✓ Problem, users and objectives are clearly stated
- ✓ In-scope and out-of-scope items are agreed
- ✓ Functional requirements are testable and prioritised
- ✓ Modules, inputs, outputs and stored data are identified
- ✓ Non-functional requirements and technology choices are stated
- ✓ Requirements document and PPT use the same scope

DECISION REQUIRED

Approved / Revise

Key changes requested:

_____

_____

NEXT MILESTONE

Project Design Document & PPT

Carry forward the updated HackerRank table.

Team Lead: Lijo P Thomas | Date: _____